

INNOPOLIS UNIVERSITY

Введение в компьютерное зрение

Илья Афанасьев, к.т.н., ЛИРС

Лекция 8 · Точки интереса: детекторы

Dr. Ilya Afanasyev, i.afanasyev@innopolis.ru

«Introduction to Computer Vision» course, Innopolis University (IU), Russia, Fall 2026

Карта курса

Блок I Изображение и его источник	Блок II Операции над изображением	Блок III Признаки	Блок IV Гомография и геометрия	Блок V Видео и движение
L1 Введение. Изображение и видео как данные	L4 Геометрические преобразования	L7 Границы, линии и робастное оценивание	L10 Гомография: оценивание и выравнивание	L13 Видео: движение и оптический поток
L2 Формирование изображения: оптика и сенсор	L5 Пространственная фильтрация	L8 Точки интереса: детекторы	L11 Модель камеры и калибровка	L14 Трекинг и распознавание объектов
L3 Цвет, гистограммы и точечные операции	L6 Частотная область и пирамиды	L9 Дескрипторы и сопоставление	L12 Стерео и эпиполярная геометрия	L15 Синтез: от двух видов к 3D. Итоги

Сегодня — L8: Точки интереса: детекторы

1

Детектирование фитч в OpenCV

извлечение признаков · особые точки · повторяемость и различимость · инвариантность · устойчивость · панорамы

Извлечение признаков (feature extraction)

Две стратегии использования данных с датчиков **для управления поведением робота**:

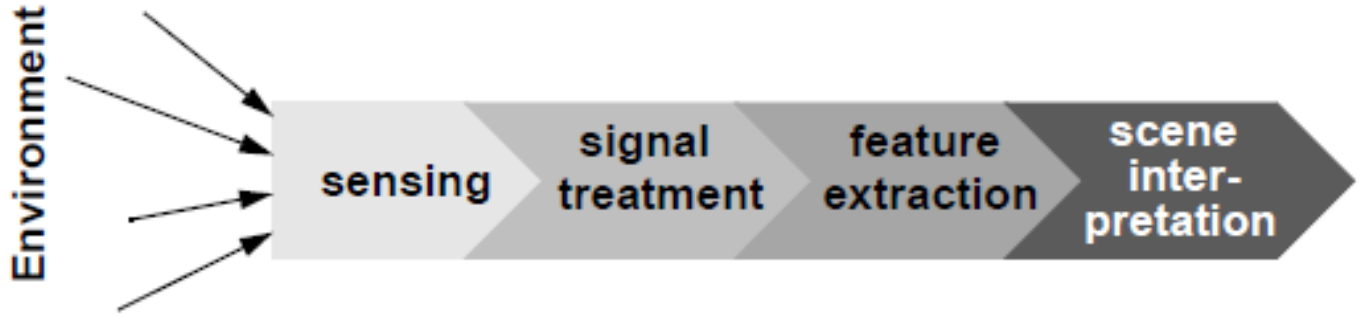
- **Прямое использование сырых данных** (rawdata): робот использует каждое измерение датчика как есть. Например, если датчик обнаруживает препятствие, робот может сразу изменить направление движения.
- **Использование промежуточной модели:** на основе сенсорных измерений создается карта окружающей среды (модель), и робот принимает решения, основываясь на этой карте. При этом сырые данные используются для обновления модели.

Извлечение признаков (feature extraction):

- **Создание высокого уровня восприятия:** включает обработку и анализ показаний датчиков для создания более **абстрактного представления о среде**. Это может включать **идентификацию объектов**, определение **их расположения** и **характеристик**.
- **Использование восприятия для управления роботом:** для обновления модели робота и принятия решений о его действиях. Это позволяет роботу лучше ориентироваться и адаптироваться к изменениям в окружающей среде.

Полный перцептивный конвейер:

- Данные с датчиков проходят через несколько этапов обработки, называемых **перцептивным конвейером**. Это включает **сбор данных**, **предварительную обработку**, **извлечение признаков** и **интерпретацию сцены**.
- **Извлечение признаков**: На этом этапе выделяются важные характеристики из данных, такие как **края**, **контуры**, **объекты** и **их расположение**.
- **Интерпретация сцены**: Полученные признаки используются для **построения модели окружающей среды**, которую робот может использовать **для навигации**.



Конвейер восприятия (**perceptual pipeline**): от показаний датчиков к моделям знаний

Fig. © Roland Siegwart, Illah R. Nourbakhsh, and Davide Scaramuzza. Introduction to Autonomous Mobile Robots, MIT press, 2011

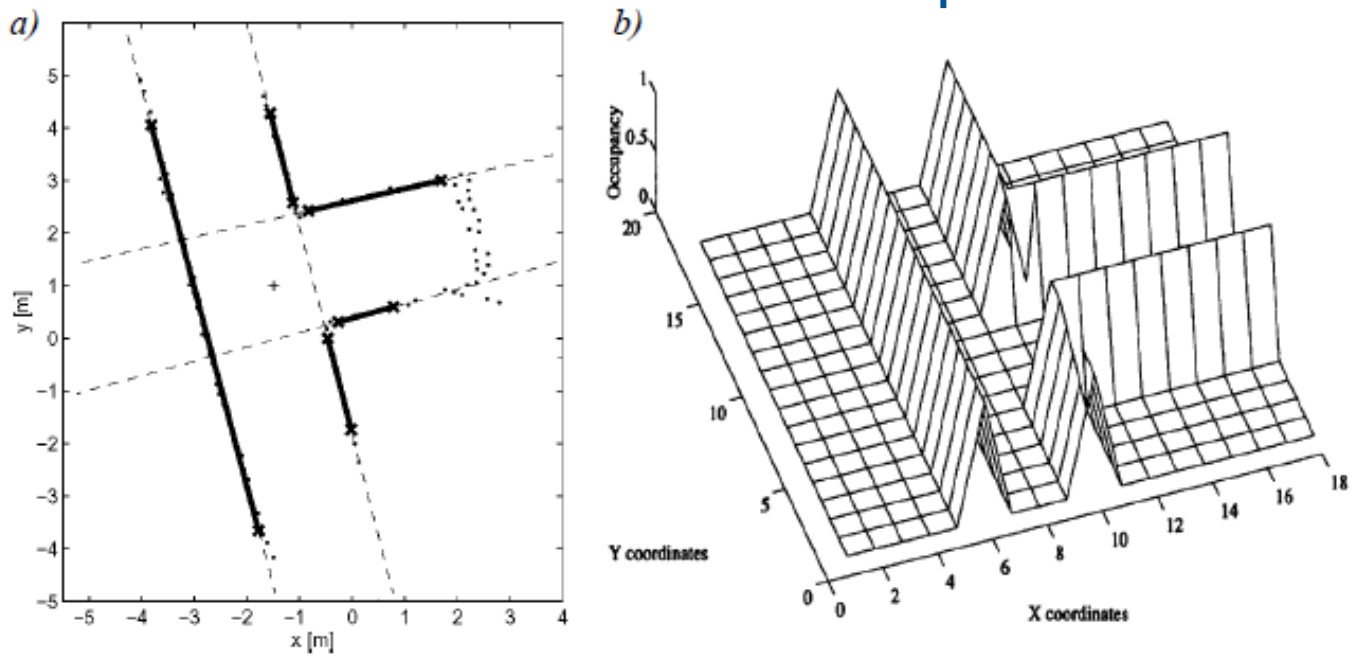
Определение признаков (Feature definition)

1. **Роль признаков в мобильной робототехнике**:
   - **Признаки** (**features**) — это **ключевые элементы**, которые роботы используют **для создания моделей окружающей среды**. Они помогают роботу **строить карты** и локализовать себя в пространстве.
2. **Типы признаков**:
   - **Низкоуровневые признаки**: Геометрические **примитивы**, такие как **линии**, **точки**, **углы**, **пятна**, **круги** и **многоугольники**. Они легко извлекаются из данных и могут быть **математически описаны**.
   - **Высокоуровневые признаки**: **Объекты**, такие как двери, столы, мусорные баки. Они обеспечивают более высокую абстракцию и точность, но **требуют больше вычислительных ресурсов** для извлечения.
3. **Необработанные данные с датчиков** (rawdata):
   - Необработанные данные содержат большой объем информации, но каждая часть данных может быть менее отличимой.
   - Преимущество использования необработанных данных заключается в максимальном сохранении информации.
4. **Извлечение признаков**:
   - **Низкоуровневые признаки**: **Фильтруют** плохие или бесполезные **данные**, **но могут привести к потере** полезной информации.
   - **Высокоуровневые признаки**: Обеспечивают **максимальную абстракцию** и **снижают объем данных**, обеспечивая высокую точность.
5. **Вычислительные затраты**:
   - Извлечение признаков, особенно визуальных, может потребовать значительных вычислительных ресурсов. Это необходимо учитывать при проектировании системы робота.

Environment representation

Два ключевых требования для визуального извлечения признаков для мобильных роботов:

- **Работа в реальном времени**: визуальная система роботов должна быстро и эффективно обрабатывать данные с датчиков для принятия решений и выполнения задач.
- **Устойчивость к условиям реального мира**: В реальных условиях освещение и обстановка могут быть непредсказуемыми и изменчивыми. Роботы должны справляться с этими изменениями. Например, **освещение может изменяться** из-за времени суток, погодных условий или других факторов, и объекты могут иметь различные цвета и текстуры. Алгоритмы извлечения признаков должны быть достаточно устойчивыми, чтобы работать в таких условиях.



Представление и моделирование окружающей среды: (a) **на основе признаков** (непрерывная метрика); (b) **на основе сетки занятости** - **occupancy grid** (дискретная метрика)

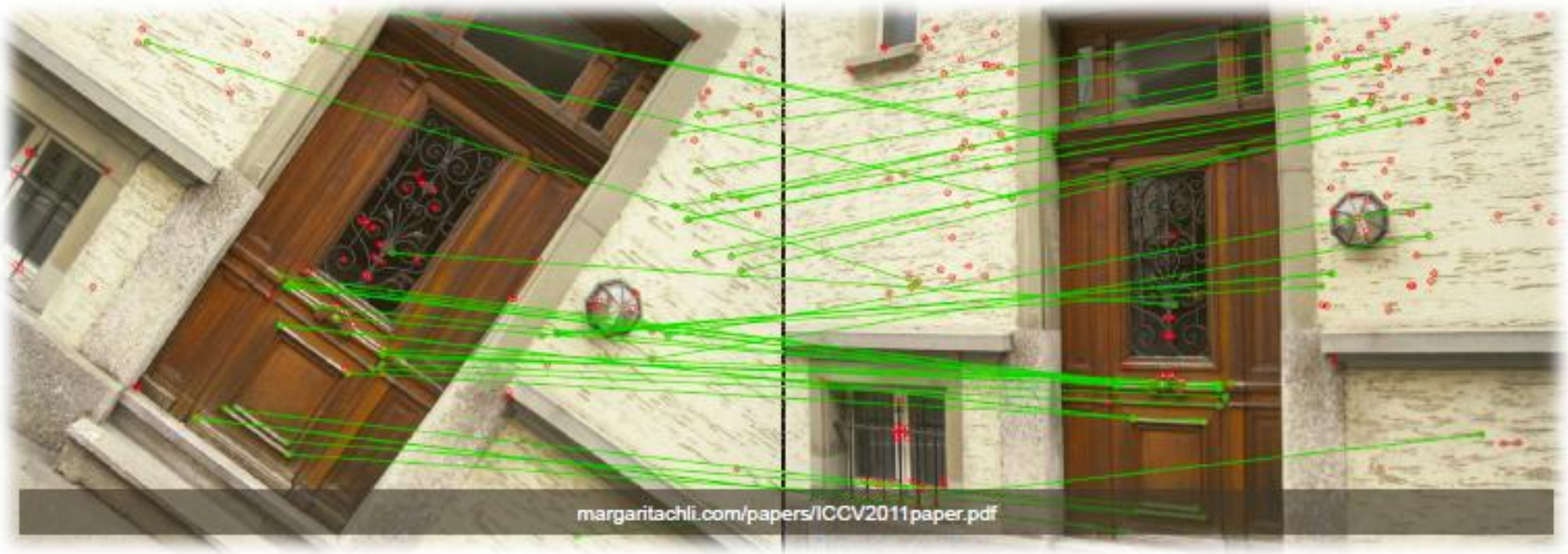
- **Поток данных с камер**: CCD (Charge-Coupled Device) камеры способны генерировать огромные объемы данных, например черно-белая камера с разрешением 3000x2000 пикселей и числом кадров в секунду fps=40 записывает **3000x2000x40x1байт = 240 Мбайт/с**.
- **Задача уменьшения объема информации**: для эффективного анализа изображения (в реальном времени).
- **Извлечение признаков**: должно четко описывать признаки и элементы окружающей среды, такие как контуры, края, объекты и их расположение.
- **Цель извлечения признаков**: преобразование большого объема необработанных данных в **компактное и информативное представление**, которое можно использовать **для построения моделей, карт и принятия решений**.

Fig. © Roland Siegwart, Illah R. Nourbakhsh, and Davide Scaramuzza. Introduction to Autonomous Mobile Robots, MIT press, 2011

Interest Point Detectors - Point Features

Image Feature Extraction:

- Edges
- **Points: Shi-Tomasi & Harris corners**
  SIFT features
  - **and more recent algorithms from the state of the art…**



© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Классификация местных признаков (local features)

1. **Местные признаки** (**Local Features**):
   - это части изображения, которые отличаются от ближайшего окружения по таким параметрам, как интенсивность, цвет и текстура. Это могут быть **небольшие области с однородным цветом**, **края** или **точки**, такие как **углы** (**corners**). Местные признаки также называют **областями интереса** или **ключевыми точками**.
2. **Категории признаков по семантическому содержанию**:
   - **Признаки с семантической интерпретацией**: Эти признаки имеют явное значение, например, **края дороги** или клетки крови на медицинских изображениях.
   - **Признаки без семантической интерпретации**: Важно, чтобы их местоположение можно было точно и надежно определять со временем. Применения включают отслеживание признаков, калибровку камер и 3D-реконструкцию.
   - **Признаки для распознавания сцен или объектов**: Они не имеют явной семантической интерпретации по отдельности, но могут использоваться для распознавания сцены или объекта в целом. Важно количество совпадений признаков между наблюдаемой сценой и эталонным изображением.
3. **Примеры применения**:
   - Извлечение признаков широко используется в различных областях, таких как **создание панорам**, **анализ текстур**, **классификация сцен**, **добыча данных из видео** и **поиск изображений в Интернете**.

Point Features: applications

Point features are widely used in:

- Robot navigation
- Object recognition
- 3D reconstruction
- Motion tracking
- Indexing and database retrieval ⇨ Google Images
- …
- Image stitching: this panorama was generated using **AUTOSTITCH** (freeware)

- Свойства идеального детектора признаков: точность, повторяемость, устойчивость, различимость, эффективность, инвариантность, простота, интегрируемость (в алгоритмы)
- Большинство современных цифровых потребительских камер оснащены программным обеспечением для автоматического создания панорам из нескольких фотографий.



© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Point Features: how to build a panorama?

- We need to match (align) images



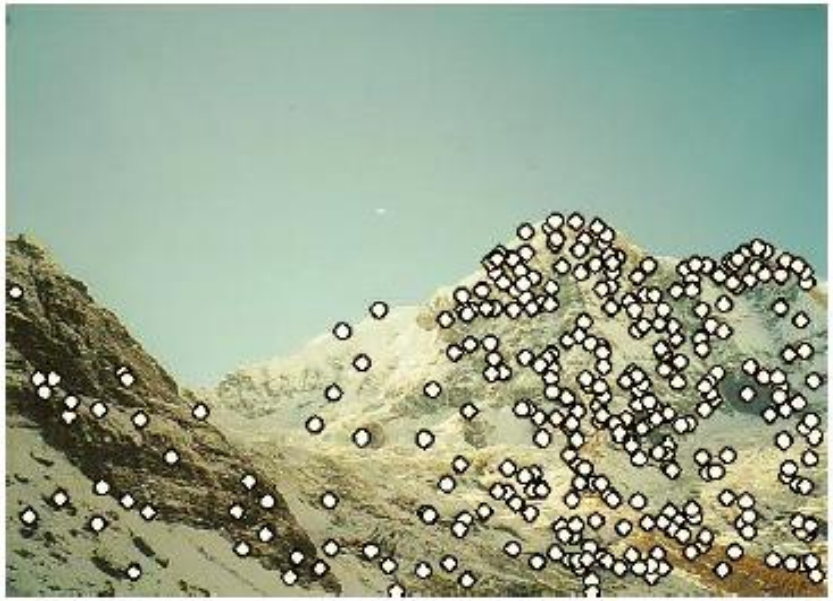


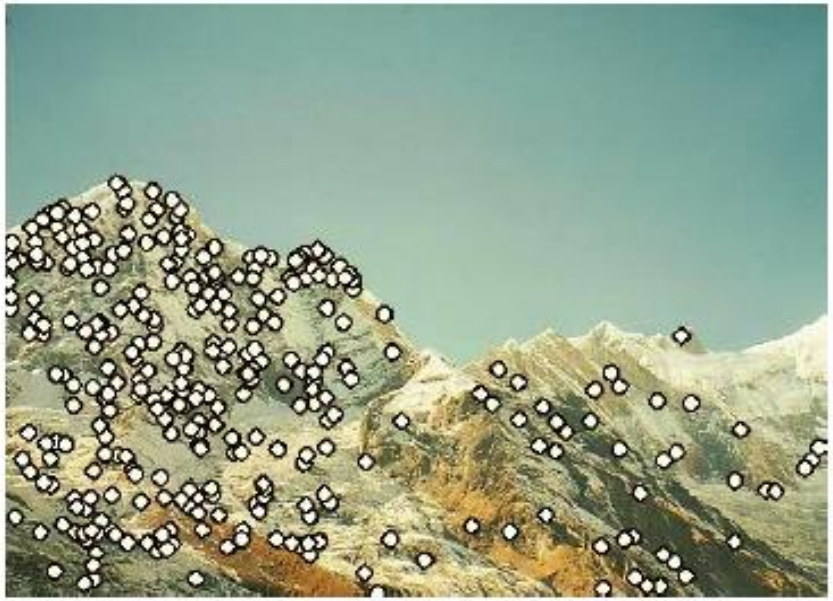
© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Point Features: how to build a panorama?

- Detect feature points in both images

The key challenge is to identify corresponding regions between overlapping images.



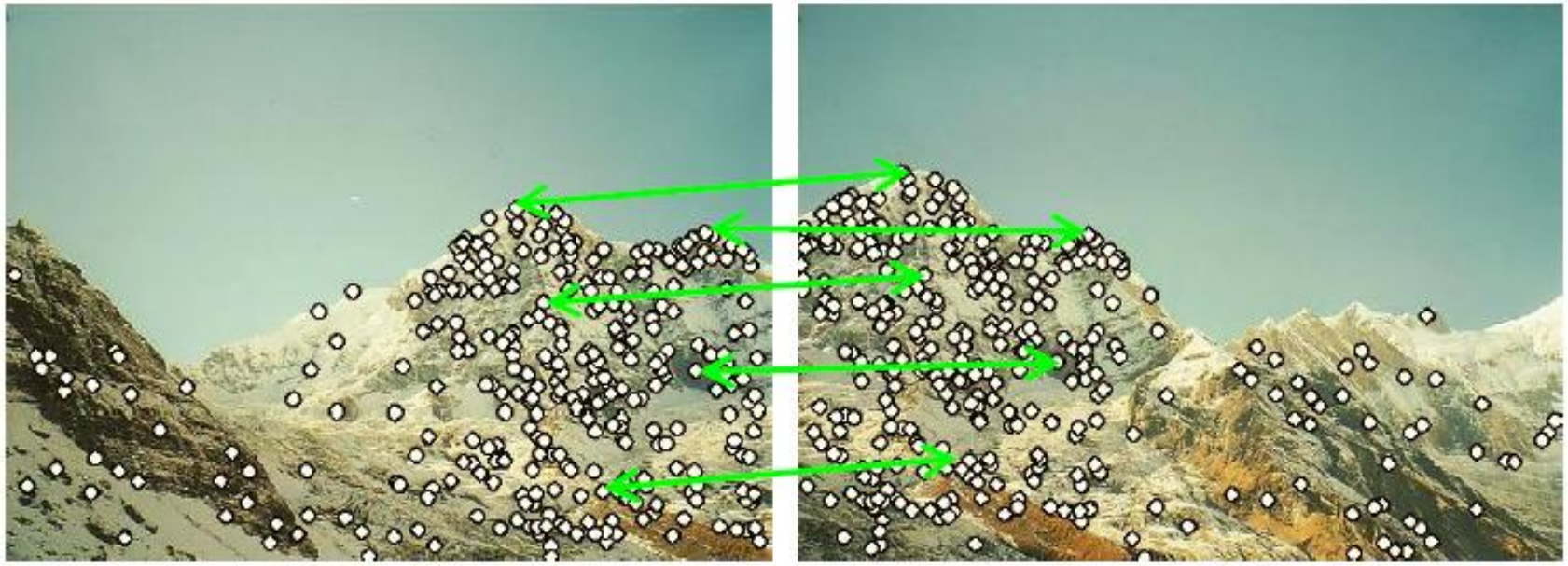


© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Point Features: how to build a panorama?

- Detect feature points in both images
- Find corresponding pairs

To extract feature points from adjacent pictures, find corresponding pairs according to some similarity measure



© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Point Features: how to build a panorama?

- Detect feature points in both images
- Find corresponding pairs
- Use these pairs to align images

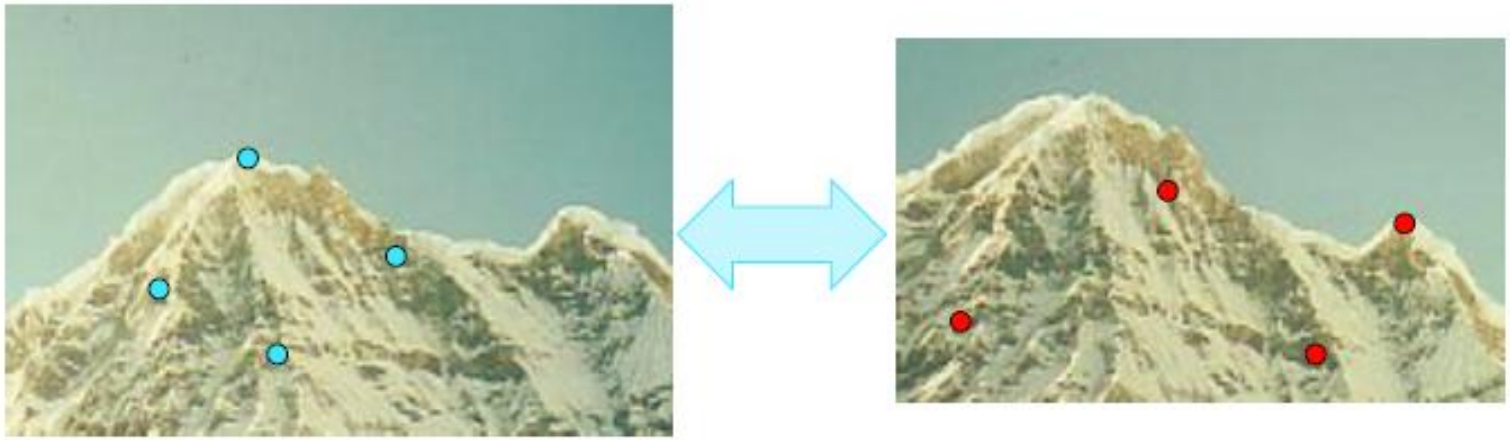
And compute the transformation (e.g., homography) to align images



© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Point Features: feature extraction

- Problem 1:
  - Detect the **same** points **independently** in both images, if they are in the field of view



no chance to match!

We need a **repeatable** feature detector

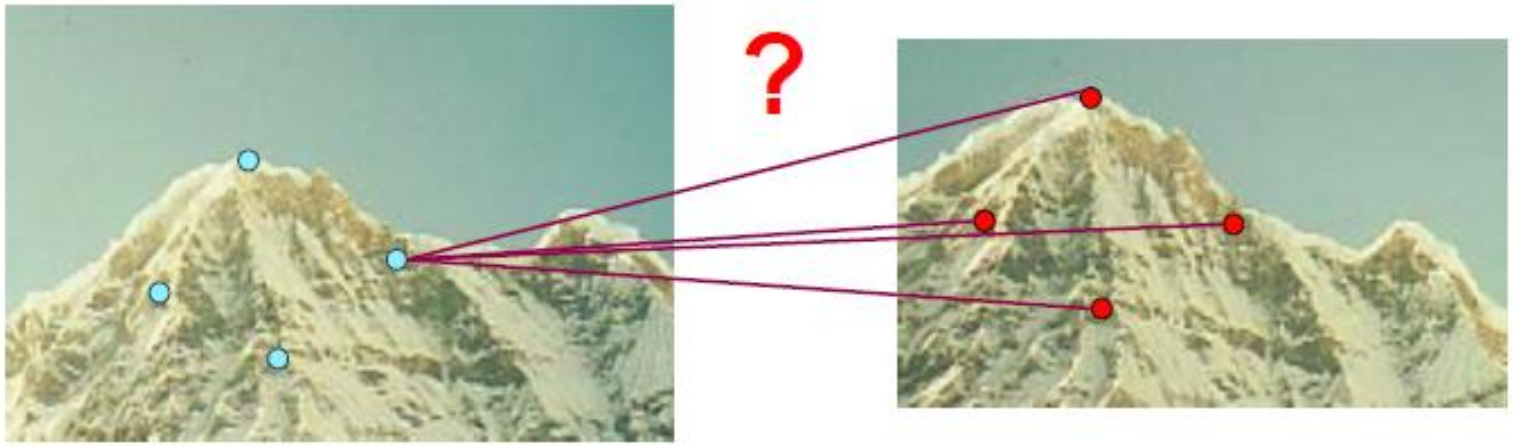
The first problem is how to detect the same points independently in both images, if the features from the left image are not redetected in the right image?

"Repeatability" is probably the most important property of a good feature detector.

© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Point Features: feature matching

- Problem 2:
  - For each point, identify its correct correspondence in the other image(s)



Thus, the feature must be invariant to view point changes, such as camera rotation or zoom (i.e., scale), and illumination changes

We need a **reliable** and **distinctive** feature descriptor

For each point in the first image we need to correctly recognize the corresponding one in the second image

Thus, the detected features should be very distinctive

For two images of the same scene taken under different viewing and illumination conditions, it is desirable that a high percent of the features can be redetected in images

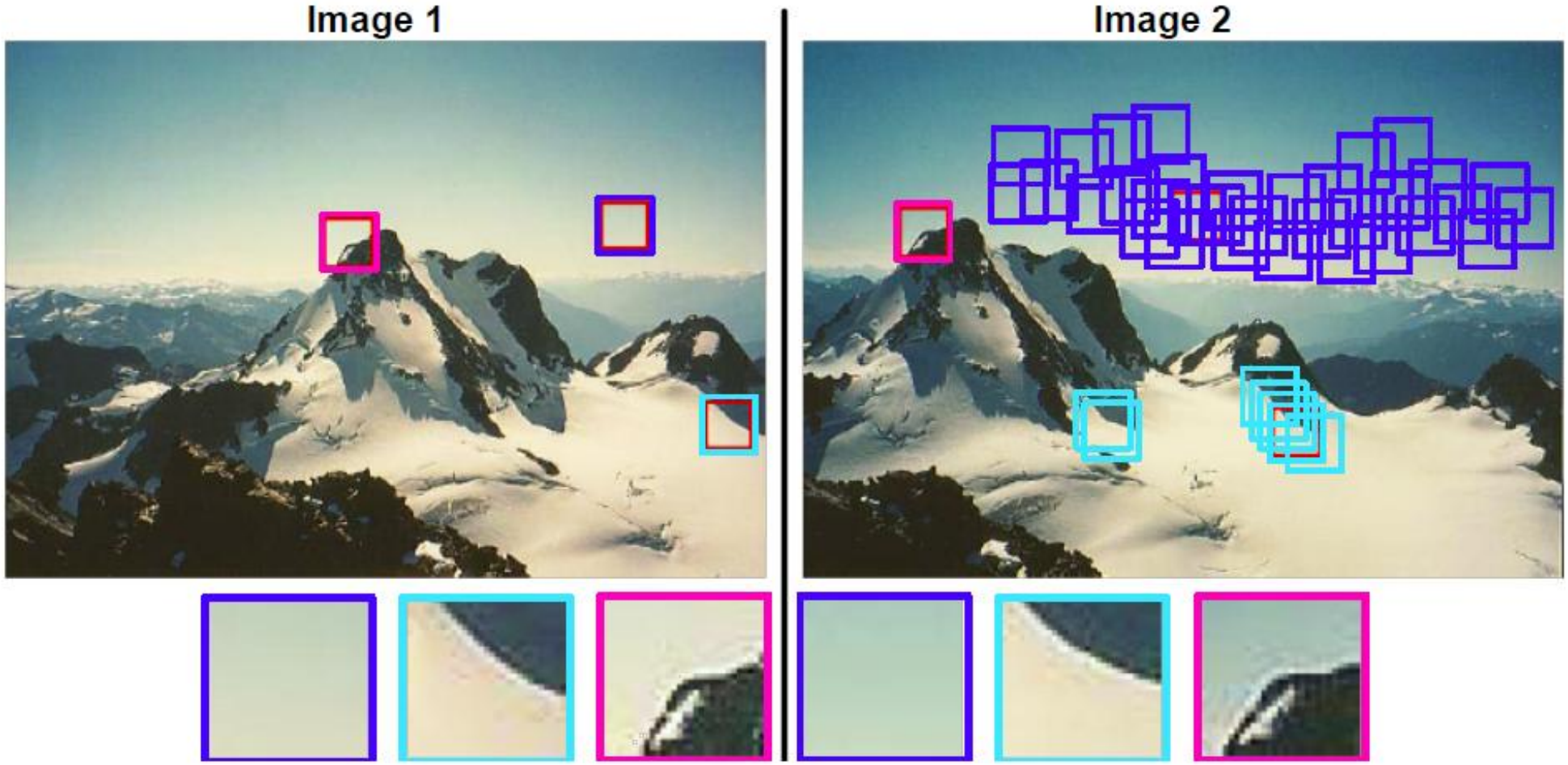
© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Point Features: what is a distinctive feature?

- Some patches can be localized or matched with higher accuracy than others

The 2nd important property is "distinctiveness" i.e., the information carried by the patch surrounding the feature point should be as distinctive as possible so that the features can be distinguished and matched.

E.g., the corners of a chessboard are not distinctive because they cannot be distinguished from each other.



© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

The properties of a good feature detector

Самые важные свойства хорошего детектора признаков (после повторяемости **repeatability** и различимости **distinctiveness**) следующие:

- **Точность локализации:** Важно, чтобы обнаруженные признаки были точно определены как по положению на изображении, так и по масштабу.
- **Количество признаков:** Идеальное количество признаков зависит от применения. Для задач распознавания объектов или сцен, поиска изображений и 3D-реконструкции требуется большое количество признаков для увеличения точности. Для задач с семантической интерпретацией достаточно небольшого количества высокоуровневых признаков.
- **Инвариантность**: Признаки должны быть инвариантны к изменениям точки обзора камеры, освещения и масштаба. Это можно достичь с помощью математического моделирования преобразований. Примером успешной инвариантности является метод SIFT.
- **Вычислительная эффективность:** Важно, чтобы признаки могли быть обнаружены и сопоставлены быстро, особенно в реальном времени. Однако время вычислений зависит от уровня инвариантности: чем больше проверок нужно выполнить, тем дольше занимает процесс.
- **Устойчивость**: Обнаруженные признаки должны быть устойчивы к различным воздействиям, таким как шум, размытость, артефакты сжатия и отклонения от математической модели. Это обеспечивает надежность и точность в реальных условиях.

2

Детектирование углов в OpenCV

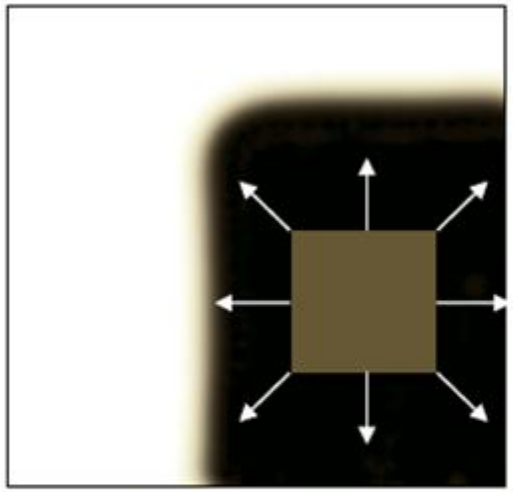
Матрица вторых моментов · собственные значения · детектор Харриса · детектор Ши-Томаси · функция угловатости (cornerness)

Corner detection: identifying corners

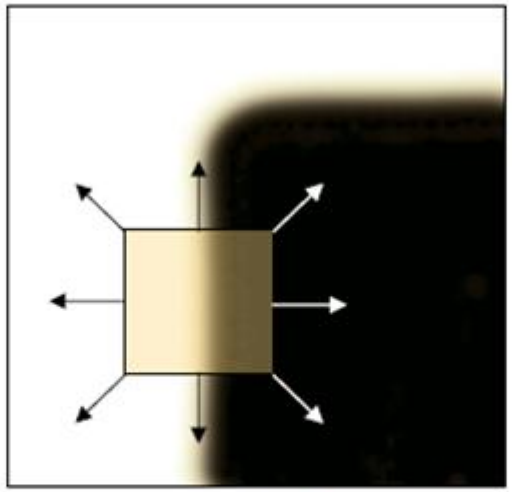
- How do we identify corners?

One of the earliest corner detectors was invented by Moravec in 1977. He defined a corner as a point where there is a large intensity variation in every direction.

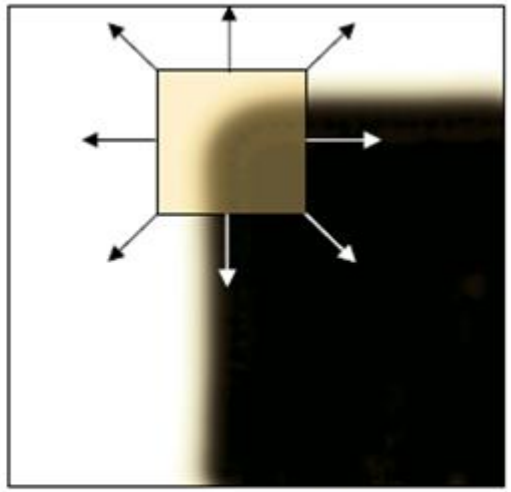
- Shifting a window in **any direction** should give a **large change** of intensity in at least 2 directions



"flat" region:
no intensity change



"edge":
no change along the edge direction



"corner":
significant change in at least 2 directions

Moravec used the Sum of Squared Differences (SSD) as a measure of the similarity between two patches. A low SSD indicates more similarity. If this number is locally maximal, then a corner is present.

© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Corner detection: how do we implement this?

- Two image patches of size P one centered at (x, y) and one centered at $(x + \Delta x, y + \Delta y)$
- The Sum of Squared Differences between them is:

$$SSD(\Delta x, \Delta y) = \sum_{x,y \in P} \left(I(x, y) - I(x + \Delta x, y + \Delta y) \right)^2$$

Intensity

Shifted Intensity

the partial derivatives of *I*

- Let $I_x = \frac{\partial I(x,y)}{\partial x}$ and $I_y = \frac{\partial I(x,y)}{\partial y}$. Approximating with a 1st order Taylor expansion:

$$I(x + \Delta x, y + \Delta y) \approx I(x, y) + I_x(x, y)\Delta x + I_y(x, y)\Delta y$$

- This produces the approximation

$$SSD(\Delta x, \Delta y) \approx \sum_{x,y \in P} \left(I_x(x, y)\Delta x + I_y(x, y)\Delta y \right)^2$$

© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Corner detection: how do we implement this?

$$SSD(\Delta x, \Delta y) \approx \sum_{x,y \in P} \left(I_x(x,y)\Delta x + I_y(x,y)\Delta y \right)^2$$

- This can be written in a matrix form as

$$SSD(\Delta x, \Delta y) \approx \sum_{x,y \in P} \begin{bmatrix} \Delta x & \Delta y \end{bmatrix} \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix} \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix}$$

$$\Rightarrow SSD(\Delta x, \Delta y) \approx \begin{bmatrix} \Delta x & \Delta y \end{bmatrix} M \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix}$$

$$M = \sum_{x,y \in P} \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

"Second moment matrix"

© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

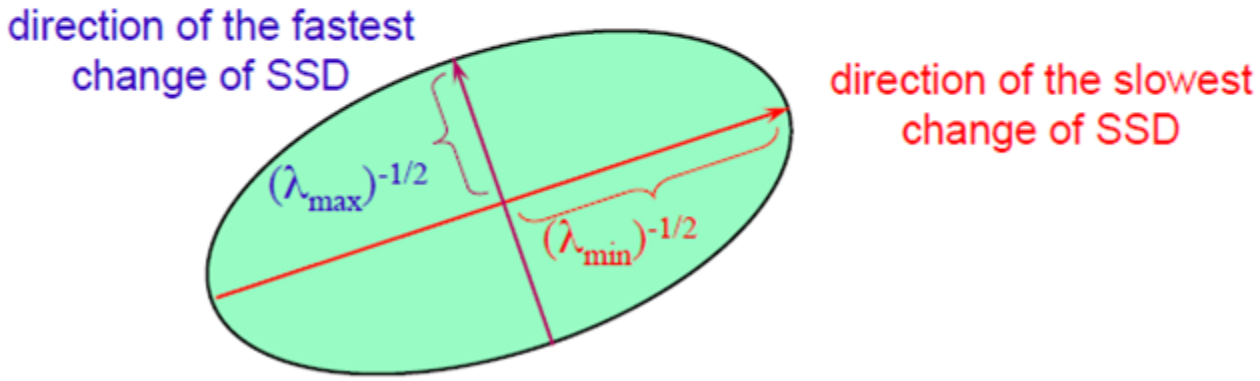
Corner detection: interpreting matrix M

- Since M is symmetric $\Rightarrow M = \sum_{x,y \in P} \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix} = R^{-1} \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} R$

- The Harris (and the Shi-Tomasi) detector analyses the **eigenvalues,** λ_1 and λ_2, to decide if we are in presence of a corner ⇨ i.e. look for large intensity changes in at least 2 directions

- We can visualize $\begin{bmatrix} \Delta x & \Delta y \end{bmatrix} M \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix} = const$ as an **ellipse** with axis-lengths determined by λ_1 and λ_2 and the axes' orientations determined by R (i.e. the **eigenvectors** of M)

- The (two) eigenvectors identify the orthogonal directions of largest and smallest changes of SSD



© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Corner detection: eigen decomposition

- The **eigenvectors** $\boldsymbol{v}$ and **eigenvalues** λ of a square matrix A satisfy:

$$A\boldsymbol{v}=\lambda\boldsymbol{v}$$

- Then is an eigenvector $\boldsymbol{v}$ of A and λ is the corresponding eigenvalue.
- The eigenvalues are found by solving: $\det(A-\lambda I)=0$
- In this case, $A=M$ is a 2 x 2 matrix, so:

$$\det\begin{bmatrix} m_{11}-\lambda & m_{12} \\ m_{21} & m_{22}-\lambda \end{bmatrix}=0$$

$$\lambda_{1,2}=\frac{1}{2}\left[(m_{11}+m_{22})\pm\sqrt{4m_{12}m_{21}+(m_{11}-m_{22})^2}\right]$$

- For each λ we can find its corresponding $\boldsymbol{v}$ (forming the columns in R) by

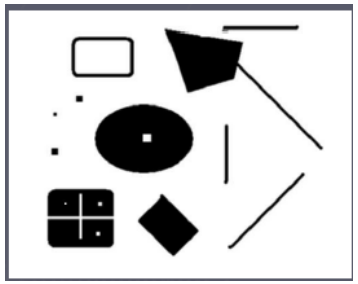
$$\begin{bmatrix} m_{11}-\lambda & m_{12} \\ m_{21} & m_{22}-\lambda \end{bmatrix}\begin{bmatrix} x \\ y \end{bmatrix}=0$$

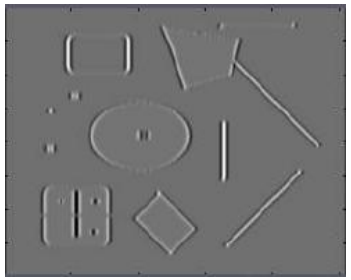
© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Corners as distinctive interest points

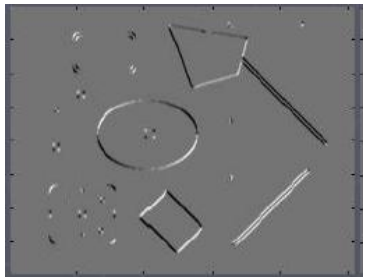
$$M = \sum w(x,y) \begin{bmatrix} I_x I_x & I_x I_y \\ I_x I_y & I_y I_y \end{bmatrix}$$

2 x 2 matrix of image derivatives
(averaged in neighborhood of a point)







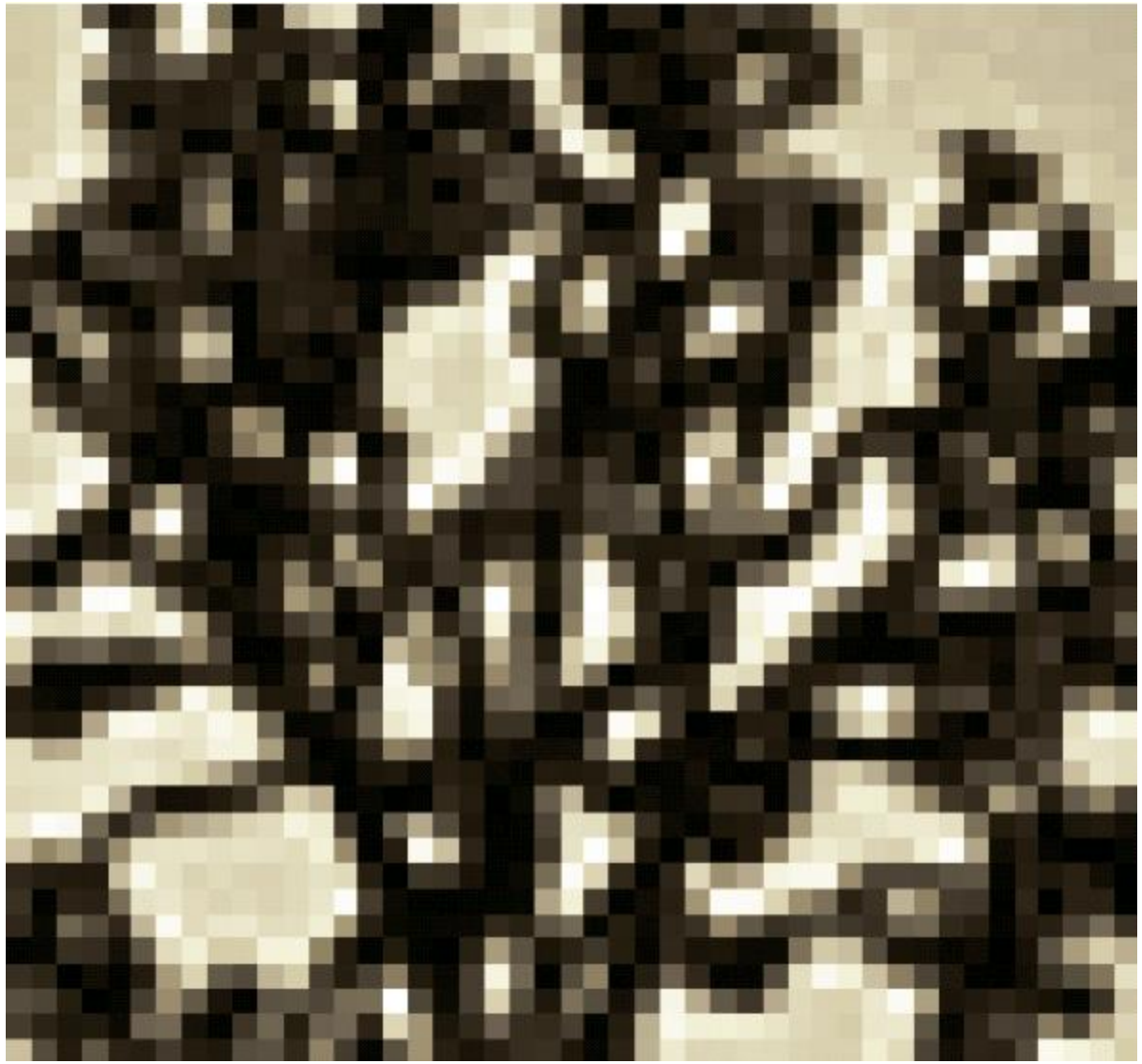


Notation:

$$I_x \Leftrightarrow \frac{\partial I}{\partial x} \qquad I_y \Leftrightarrow \frac{\partial I}{\partial y} \qquad I_x I_y \Leftrightarrow \frac{\partial I}{\partial x}\frac{\partial I}{\partial y}$$

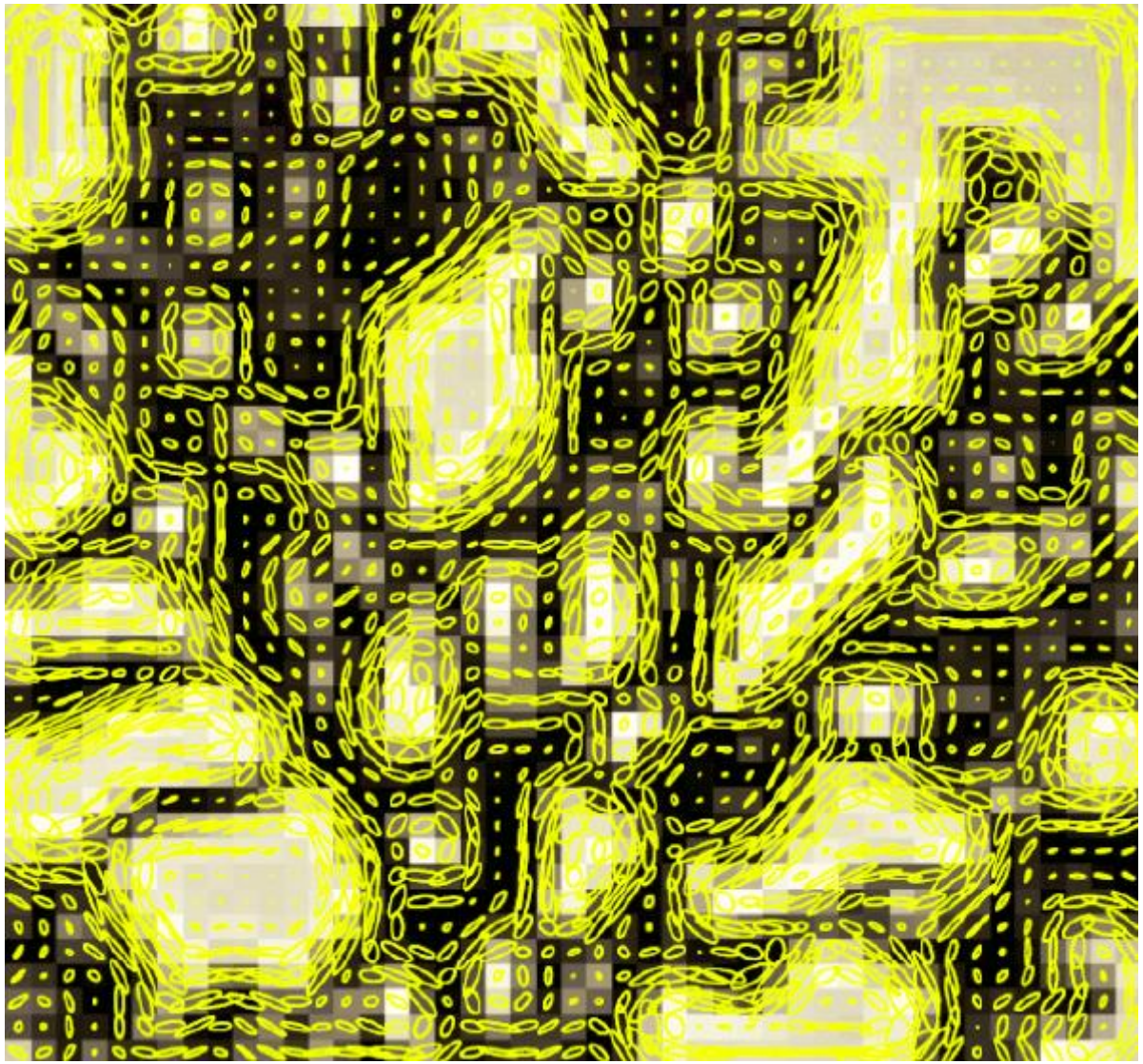
© James Hays

Corner detection: visualization of 2nd moment matrices



© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Corner detection: visualization of 2nd moment matrices



© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

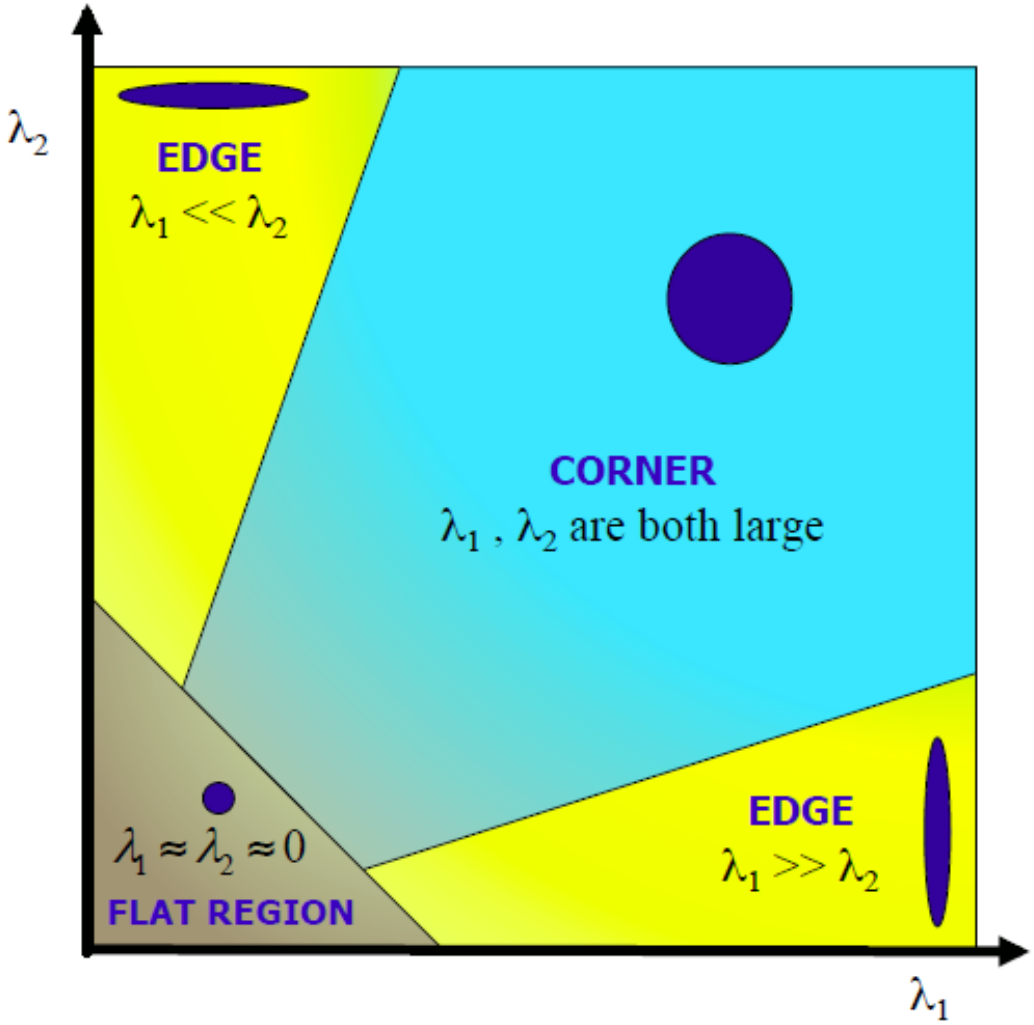
Corner detection: corner response function

- Computing λ_1 and λ_2 is expensive
  ⇨ Harris & Stephens suggested using a "**cornerness function**" instead:

$$C = \lambda_1\lambda_2 - \kappa(\lambda_1 + \lambda_2)^2 = \det(M) - \kappa \cdot trace^2(M)$$

where $\kappa = const. \in [0.04, 0.15]$

- **Harris** cornerness criterion [2]
- Last step of Harris corner detector: extract local maxima of the cornerness function



[2] C.Harris and M.Stephens. "A Combined Corner and Edge Detector." , *Proceedings of the Alvey Vision Conference, 1988*

© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Corner detection



© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Corner detection: identifying corners

- Key property: in the region around a corner, image gradient has **two or more** dominant directions
- Corners are **repeatable** and **distinctive**

Many of the features detected in the left image have also been redetected in the right image. This means that the repeatability of the Harris detector under rotations and small changes of illumination is high.



© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

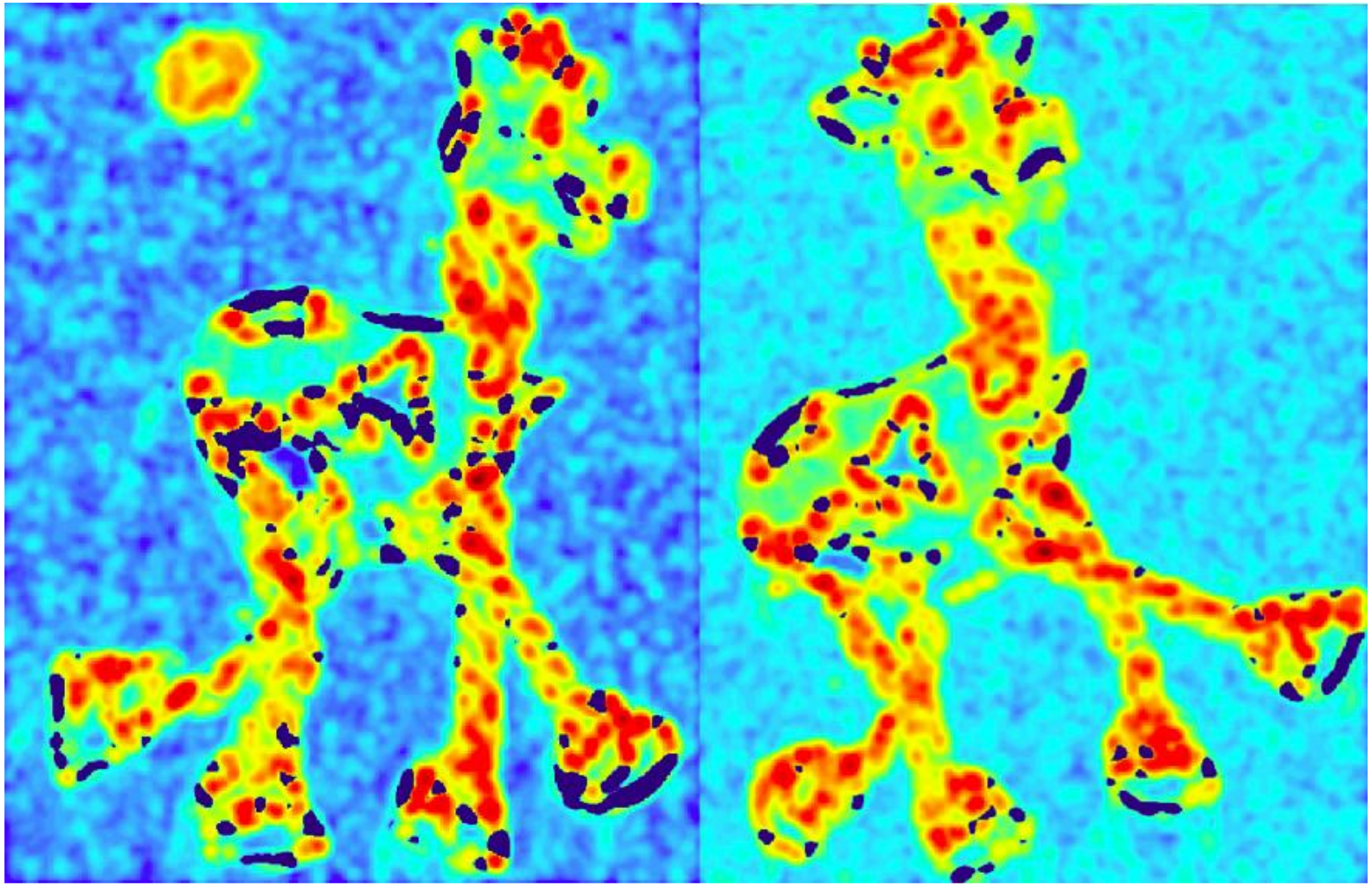
Harris corners: workflow



The images are related by a rotation and a change of illumination

© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Harris corners: workflow



Evaluation of Cornerness function C

- Compute corner response C

© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

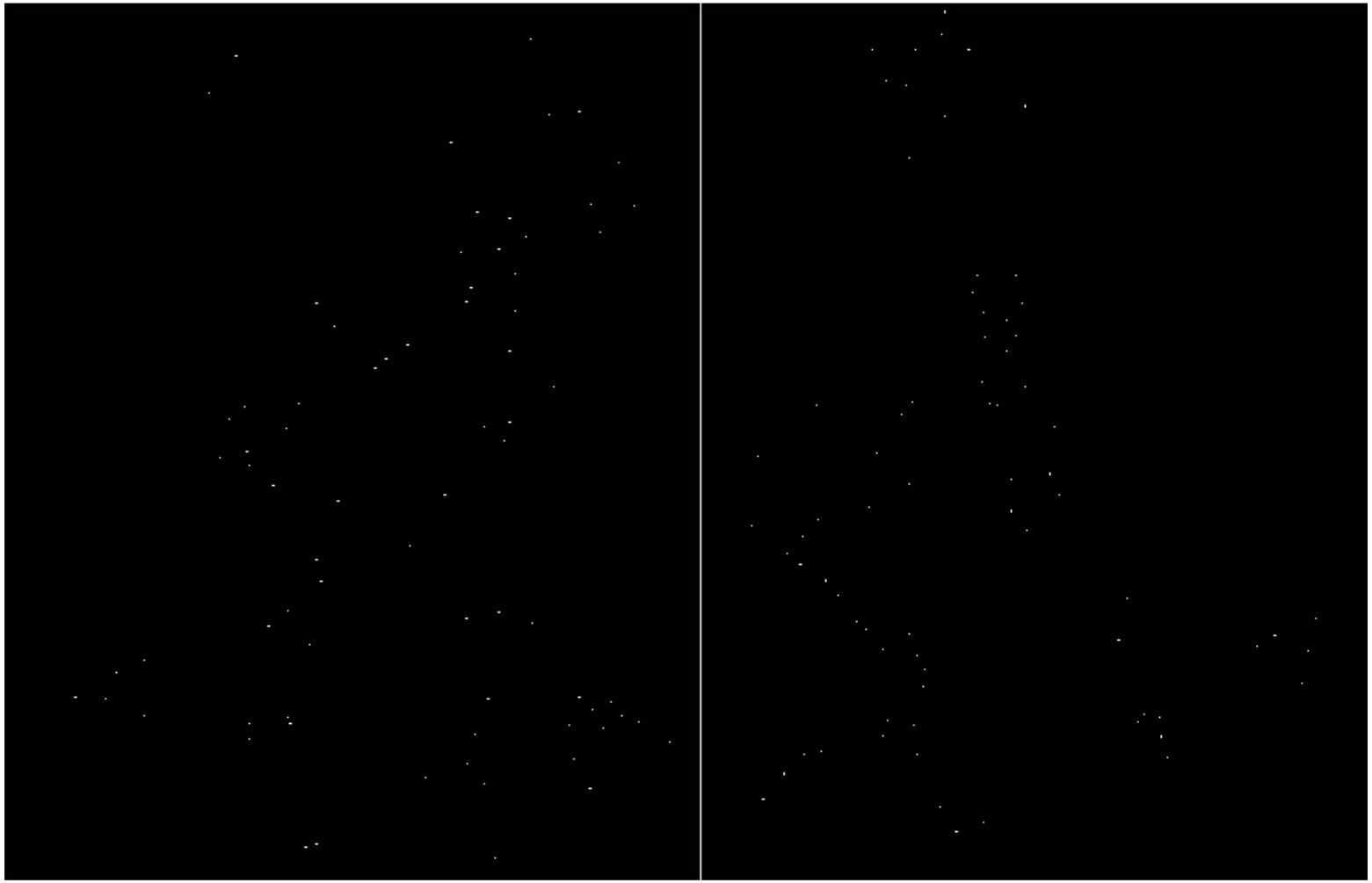
Harris corners: workflow



- Find points with large corner response: $C > threshold$

© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Harris corners: workflow



- Take only the points of local maxima of thresholded C

© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

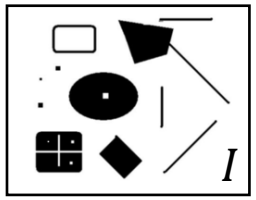
Harris corners: workflow



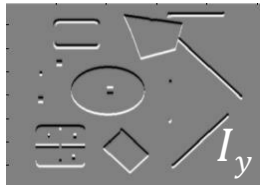
Harris corners are identified as local maxima of the cornerness function (only local maxima larger than a given threshold are retained – marked as red points)

© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

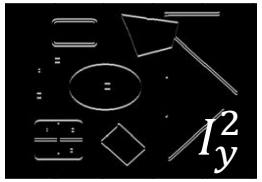
Harris Corner Detector [Harris88]

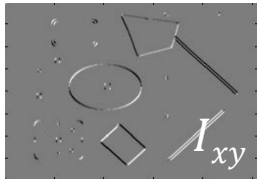






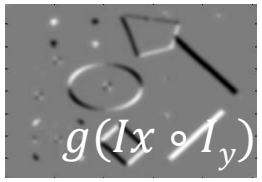














0. Input image
   We want to compute M at each pixel.

1. Compute image derivatives (optionally, blur first).

2. Compute M components
   as squares of derivatives.

3. Gaussian filter *g()* with width σ

4. Compute cornerness

$$C = \det(M) - \alpha \operatorname{trace}(M)^2$$
$$= g(I_x^2) \circ g(I_y^2) - g(I_x \circ I_y)^2 - \alpha\left[g(I_x^2) + g(I_y^2)\right]^2$$

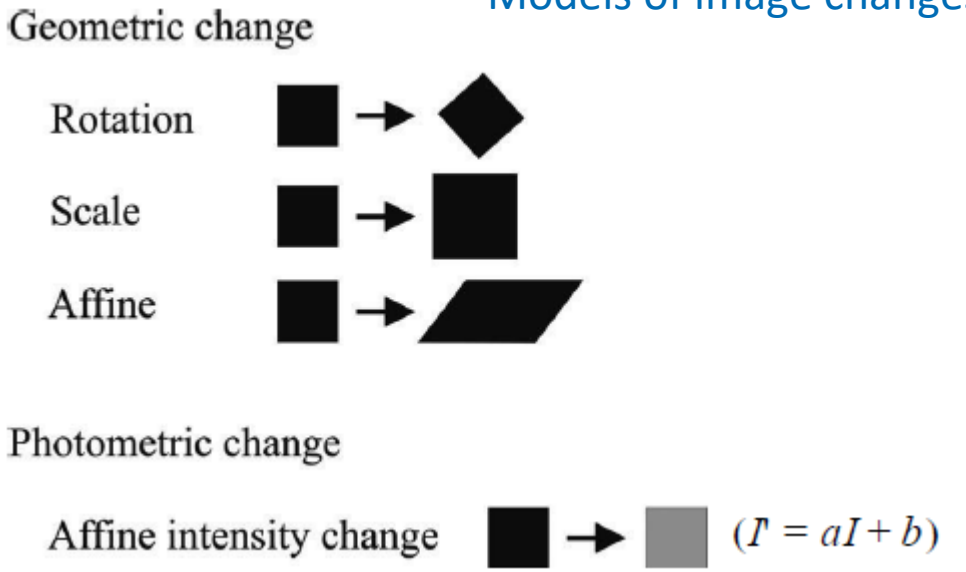
5. Threshold on C to pick high cornerness

6. Non-maxima suppression to pick peaks.

Invariance to photometric and geometric changes

- We want features to be detected despite geometric and photometric changes in the image: if we have two transformed versions of the same image, features should be detected in corresponding locations.
- Image transformations can affect the geometric or the photometric properties of an image.
- Consolidated models of image transformations are the following:
  - Geometric changes:
    - 2D Rotation
    - Scale (uniform rescaling)
    - Affine
  - Photometric changes
    - Affine intensity

Models of image changes

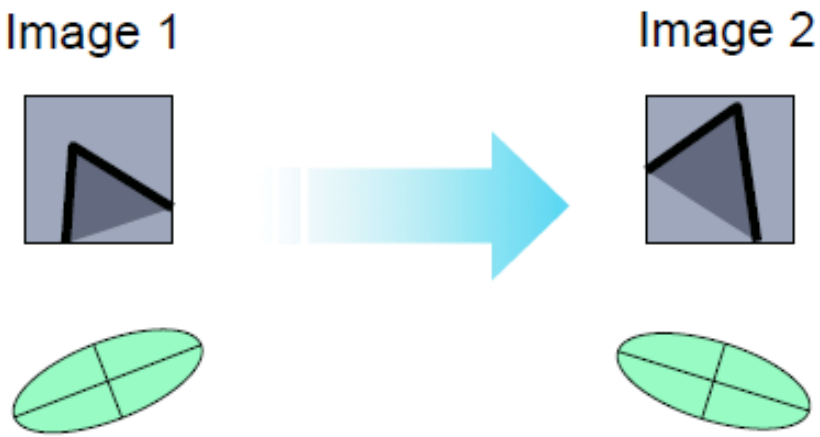


We did not mention changes of camera viewpoint (i.e., perspective distortions). In this case the transformation valid only for planar objects would be a homography. However, when the viewpoint changes are small and the object is locally planar, the affine transformation is a good approximation of the homography.

© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Harris corners: properties

- Rotation invariance?



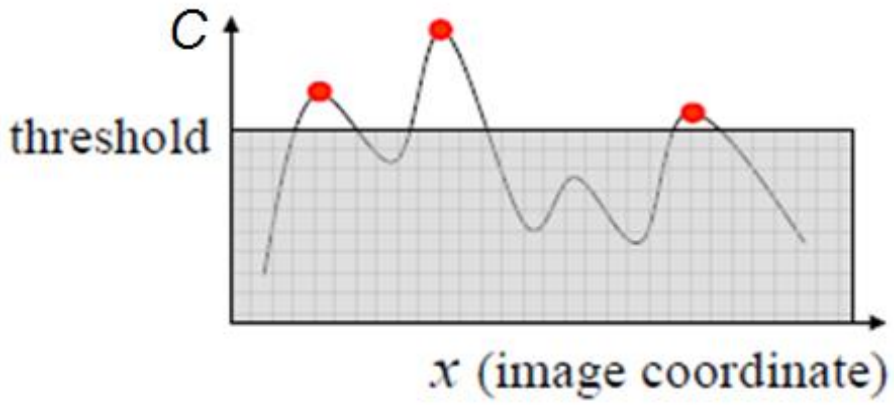
Ellipse rotates, but its shape (i.e. eigenvalues) remains the same

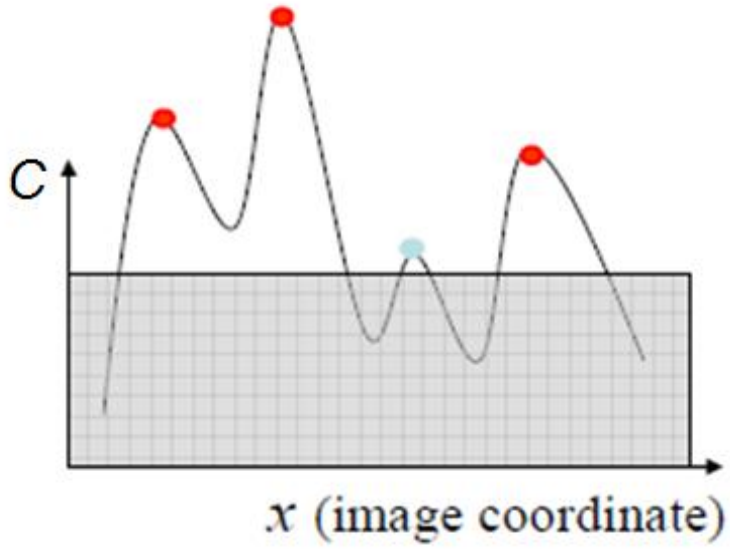
Harris corners are **invariant to image rotation**

© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Harris corners: properties

- Partial invariance to *affine intensity* change
  - ☒ Only derivatives are used => invariance to intensity shift $I \rightarrow I + b$
  - ☒ Intensity scale: $I \rightarrow a\, I$





Corner response C is invariant to linear & partially affine intensity changes

The Harris detector is invariant to affine intensity changes due to the eigenvalues, and so the cornerness function, are rescaled by a constant factor. But the position of the local maxima of the cornerness function remains the same.

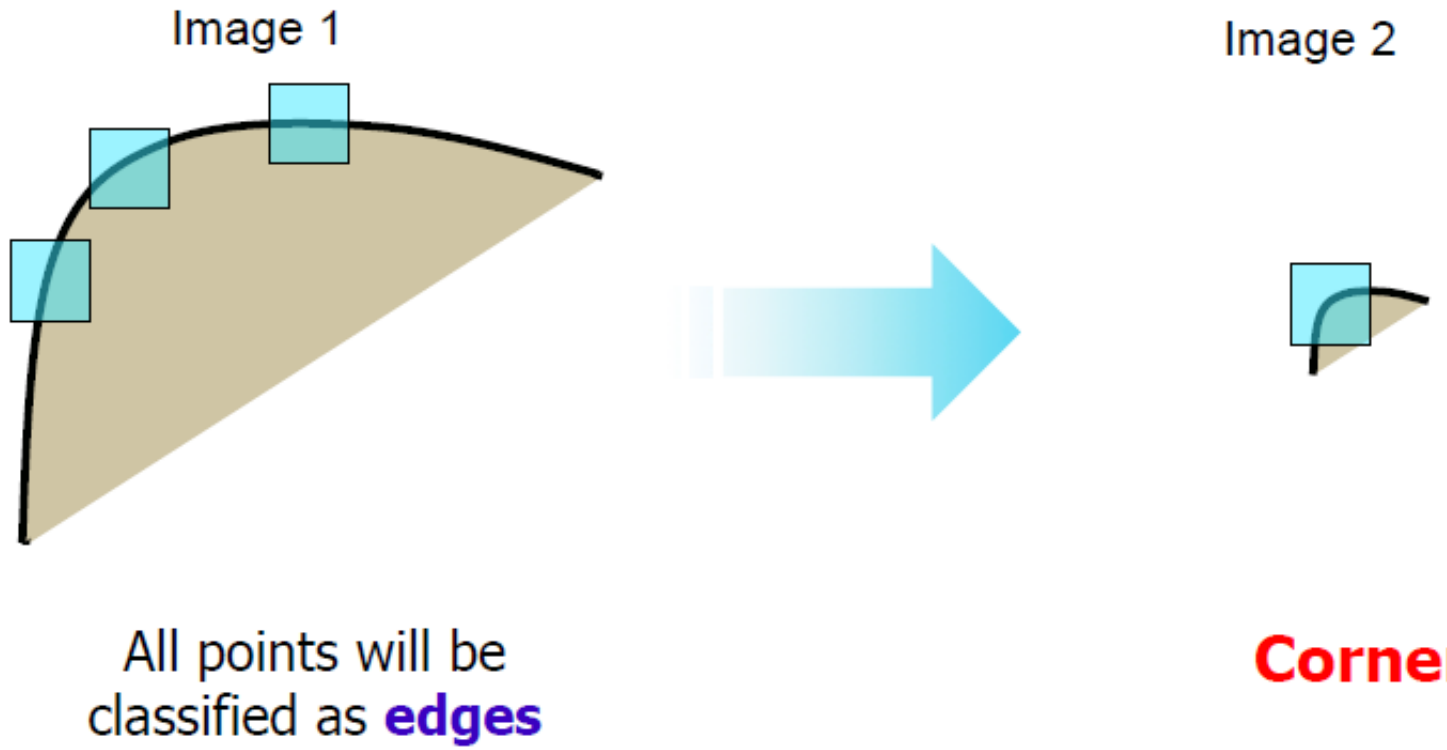
© Rick Szeliski, Lecture notes on the Harris Detector, CSE576, University of Washington, 2005

Harris corners: properties

- Scale invariance?

Harris detector is non-invariant to image scale!

Harris detector is not invariant to geometric affine transformations or scale changes. Intuitively, an affine transformation distorts the neighborhood of the feature along the x and y directions and, accordingly, a corner can get reduced or increased its curvature.



Harris corners are **not invariant to scale change**

© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

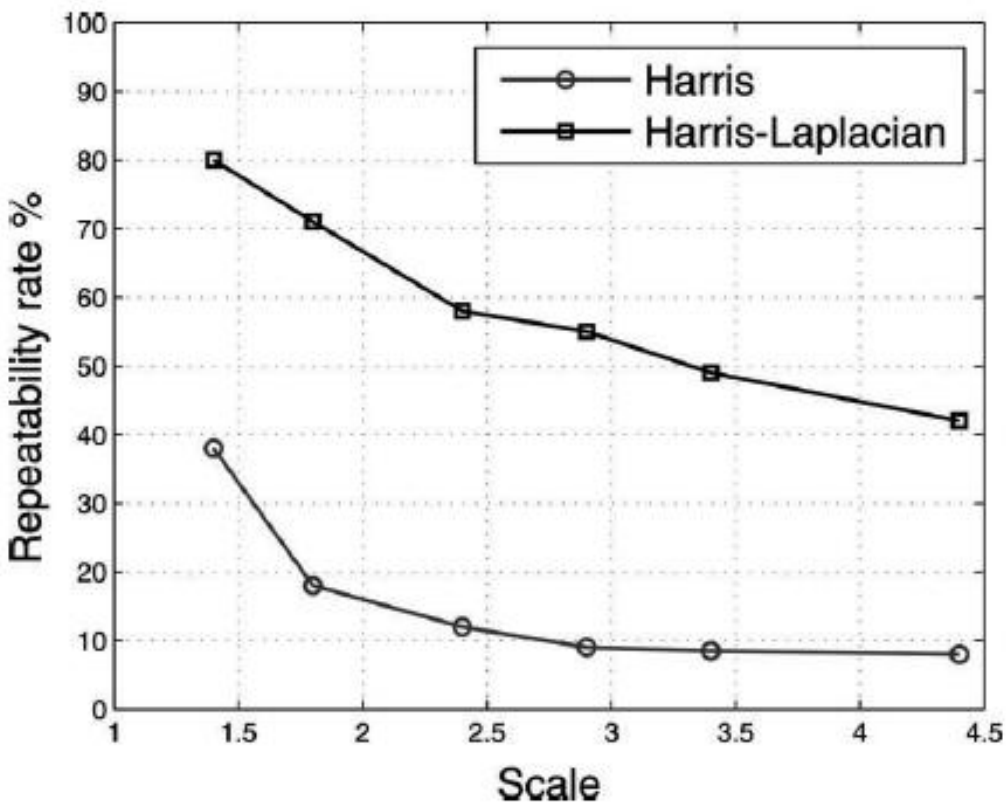
Harris corners: properties' summary

- Harris detector: probably the most widely used and known corner detector
- The detection is invariant to
  - Rotation
  - Linear intensity changes
  - **note:** to make the matching invariant to these we need suitable descriptor & matching
- The detection is NOT invariant to
  - Scale changes
  - Geometric affine changes: an image transformation, which distorts the neighborhood of the corner, can distort its '*cornerness*' response

© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Repeatability rate of Harris detector

- The performance of the Harris detector against scale changes demonstrates that the repeatability rate falls with the growth of the scale factor.
- The repeatability rate between two images is computed as the ratio between the number of found correspondences and the number of all possible correspondences.
- After rescaling the image by a factor 2, only 20% of the possible correspondences are redetected.
- Although it is not invariant to scale changes, it is still widely used and can be found in the well-known Intel open-source computer vision library (OpenCV).
- Some modifications to the original implementation have made the Harris detector also invariant to scale and affine changes.
- Additionally, the location accuracy of the Harris corners can be improved up to subpixel precision. This can be achieved by approximating the cornerness function in the neighborhood of a local maximum through a quadratic function.

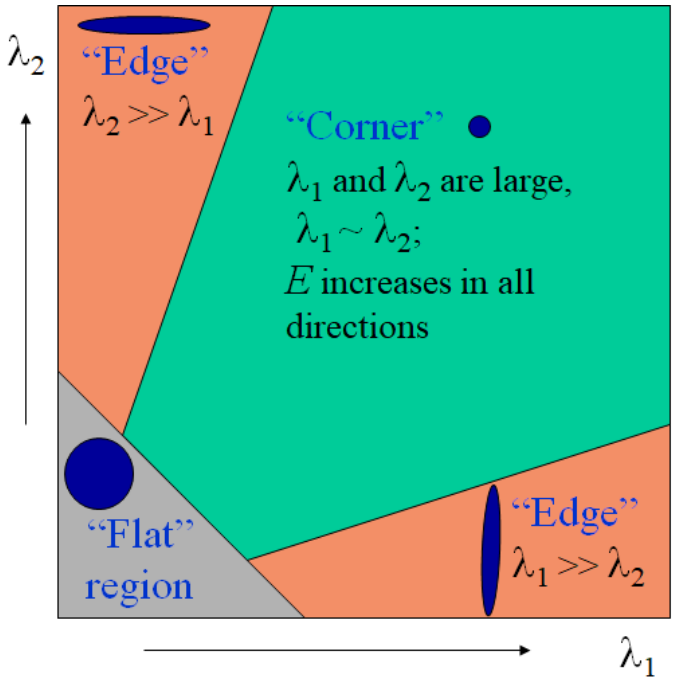


Repeatability rate of Harris detector. Comparison with Harris-Laplacian.

Harris Corner Detector in OpenCV

- in 1988, Chris Harris & Mike Stephens published this detector in their paper “A Combined Corner and Edge Detector”, so now it is called the Harris Corner Detector. It basically finds the difference in intensity for a displacement of (u,v) in all directions:

$$E(u,v) = \sum_{x,y} \underbrace{w(x,y)}_{\text{window function}} \underbrace{[I(x+u,y+v)}_{\text{shifted intensity}} - \underbrace{I(x,y)}_{\text{intensity}}]^2$$



- The window function is either a rectangular window or a Gaussian window which gives weights to pixels underneath.
- We have to maximize this function E(u,v) for corner detection $E(u,v) \approx \begin{bmatrix} u & v \end{bmatrix} M \begin{bmatrix} u \\ v \end{bmatrix}$

where $M = \sum_{x,y} w(x,y) \begin{bmatrix} I_x I_x & I_x I_y \\ I_x I_y & I_y I_y \end{bmatrix}$

- Here, Ix and Iy are image derivatives in x and y directions respectively. These can be found using cv.Sobel().
- Then comes the main part. After this, they created a score, basically an equation, which determines if a window can contain a corner or not.

$$R = \det(M) - k(\text{trace}(M))^2 = \lambda_1\lambda_2 - k(\lambda_1 + \lambda_2)^2$$

- So the magnitudes of these eigenvalues decide whether a region is a corner, an edge, or flat.

- $\det(M) = \lambda_1\lambda_2$
- $\text{trace}(M) = \lambda_1 + \lambda_2$
- λ_1 and λ_2 are the eigenvalues of M

© Harris Corner Detection, OpenCV docs, 2022, https://docs.opencv.org/4.x/dc/d0d/tutorial_py_features_harris.html

Harris Corner Detector in OpenCV

Function	Description
cv2.cornerHarris()	runs Harris corner detector on an image, computing for each pixel (x,y) a 2x2 gradient covariance matrix $M^{(x,y)}$ over a blockSize x blockSize neighborhood. Then it calculates $dst(x,y)=detM^{(x,y)}-k\cdot(trM^{(x,y)})^2$ Corners in the image can be found as the local maxima of this response map

```python
import numpy as np
import cv2 as cv

filename = 'chessboard.png'
img = cv.imread(filename)
gray = cv.cvtColor(img,cv.COLOR_BGR2GRAY)

gray = np.float32(gray)
dst = cv.cornerHarris(gray,2,3,0.04)

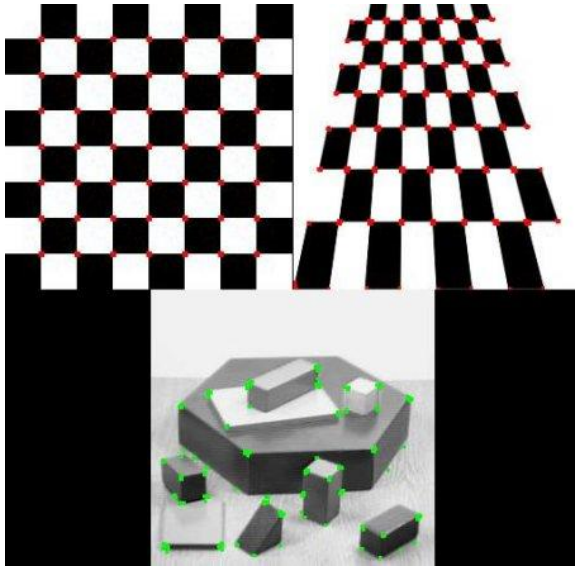
#result is dilated for marking the corners, not important
dst = cv.dilate(dst,None)

# Threshold for an optimal value, it may vary depending on the image.
img[dst>0.01*dst.max()]=[0,0,255]

cv.imshow('dst',img)
if cv.waitKey(0) & 0xff == 27:
    cv.destroyAllWindows()
```

cv2.cornerHarris() parameters:

- **img** - Input image. It should be grayscale and float32 type.
- **blockSize** - It is the size of neighbourhood considered for corner detection
- **ksize** - Aperture parameter of the Sobel derivative used.
- **k** - Harris detector free parameter in the equation.



© Harris Corner Detection, OpenCV docs, 2022, https://docs.opencv.org/4.x/dc/d0d/tutorial_py_features_harris.html

Harris Corner Detector with SubPixel Accuracy

Function	Description
cv2.cornerSubPix()	refines the corner locations with sub-pixel accuracy

```
import numpy as np
import cv2 as cv

filename = 'chessboard2.jpg'
img = cv.imread(filename)
gray = cv.cvtColor(img,cv.COLOR_BGR2GRAY)

# find Harris corners
gray = np.float32(gray)
dst = cv.cornerHarris(gray,2,3,0.04)
dst = cv.dilate(dst,None)
ret, dst = cv.threshold(dst,0.01*dst.max(),255,0)
dst = np.uint8(dst)

# find centroids
ret, labels, stats, centroids = cv.connectedComponentsWithStats(dst)

# define the criteria to stop and refine the corners
criteria = (cv.TERM_CRITERIA_EPS + cv.TERM_CRITERIA_MAX_ITER, 100, 0.001)
corners = cv.cornerSubPix(gray,np.float32(centroids),(5,5),(-1,-1),criteria)

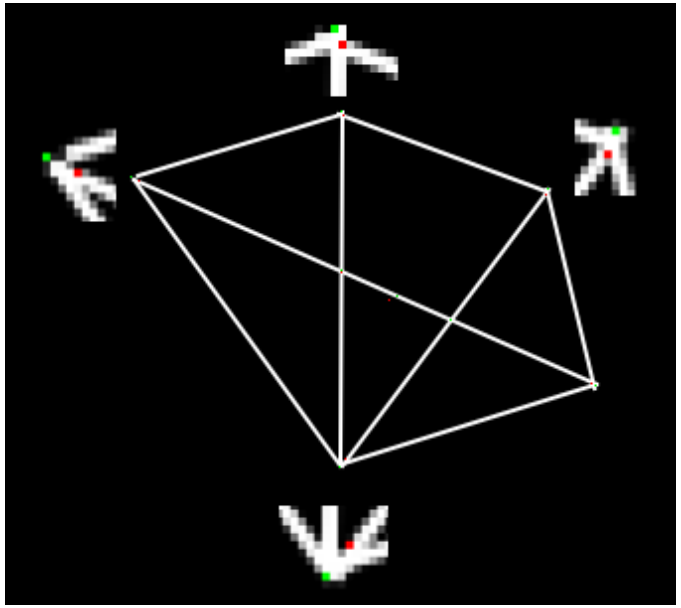
# Now draw them
res = np.hstack((centroids,corners))
res = np.int0(res)
img[res[:,1],res[:,0]]=[0,0,255]
img[res[:,3],res[:,2]] = [0,255,0]

cv.imwrite('subpixel5.png',img)
```

cv2.cornerSubPix() parameters:

- **img** - Input image. It should be grayscale and float32 type.
- **corners** - Initial coordinates of the input corners and refined coordinates provided for output
- **winSize** - Half of the side length of the search window. For example, if winSize=Size(5,5) , then a (5∗2+1)×(5∗2+1)=11×11 search window is used.
- **zeroZone** - The value of (-1,-1) indicates that there is no dead region
- **criteria** - Criteria for termination of the iterative process of corner refinement.

- In code we find the Harris corners first.
- Then we pass the centroids of these corners (There may be a bunch of pixels at a corner, we take their centroid) to refine them.
- Harris corners are marked in red pixels and refined corners are marked in green pixels.
- Then we define the criteria when to stop the iteration. We stop it after a specified number of iterations or a certain accuracy is achieved.
- We also define the size of the neighbourhood it searches for corners.



© Harris Corner Detection, OpenCV docs, 2022, https://docs.opencv.org/4.x/dc/d0d/tutorial_py_features_harris.html

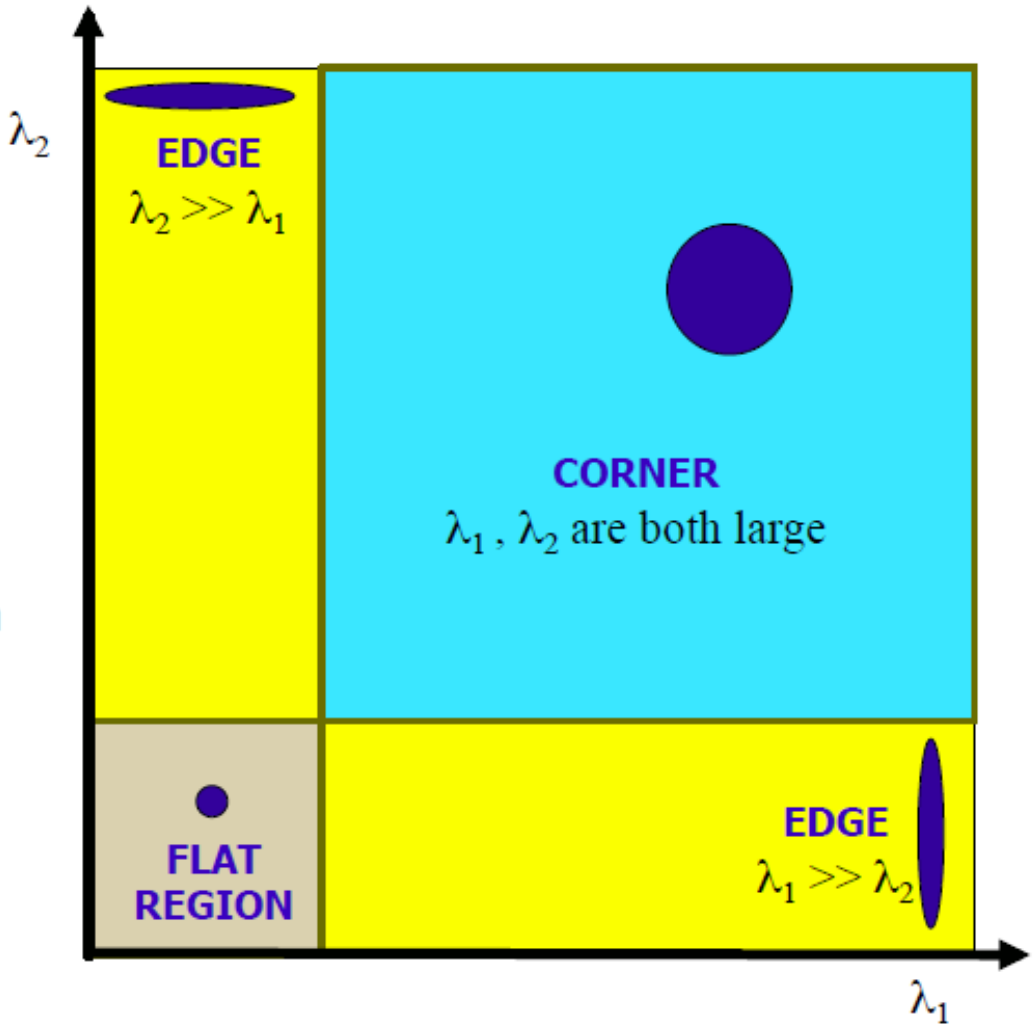
Shi-Tomasi Corner detector

Does patch P describe a corner or not? $M = R^{-1}\begin{bmatrix}\lambda_1 & 0\\ 0 & \lambda_2\end{bmatrix}R$

- **No structure:** $\lambda_1 \approx \lambda_2 \approx 0$
  SSD is almost constant in all directions, so it's a **flat** region
- **1D structure:** $\lambda_1 >> \lambda_2$ is large (or vice versa)
  SSD has a large variation only in one direction, which is the one perpendicular to the **edge**.
- **2D structure:** λ_1, λ_2 are both large
  SSD has large variations in all directions and then we are in presence of a **corner**.
- **Shi-Tomasi** [1] cornerness criterion:

$$C = min(\lambda_1, \lambda_2) > thresh.$$

[1] J. Shi and C. Tomasi. "Good Features to Track,". CVPR 1994



© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Shi-Tomasi Corner Detector in OpenCV

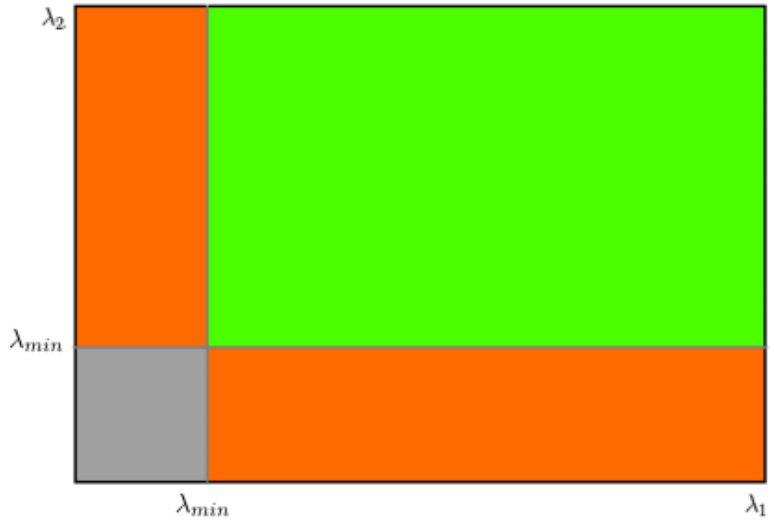
- in 1994, J. Shi and C. Tomasi made a small modification to it in their paper Good Features to Track which shows better results compared to Harris Corner Detector. The scoring function in Harris Corner Detector was given by:

$$R = \lambda_1 \lambda_2 - k(\lambda_1 + \lambda_2)^2$$

- instead of this, Shi-Tomasi proposed:

$$R = \min(\lambda_1, \lambda_2)$$

- if R is a greater than a threshold value, it is considered as a corner (green region)



Function	Description
cv2.goodFeaturesToTrack()	finds N strongest corners in the image by Shi-Tomasi method (or Harris Corner Detection, if you specify it)

```
import numpy as np
import cv2 as cv
from matplotlib import pyplot as plt

img = cv.imread('blox.jpg')
gray = cv.cvtColor(img,cv.COLOR_BGR2GRAY)

corners = cv.goodFeaturesToTrack(gray,25,0.01,10)
corners = np.int0(corners)

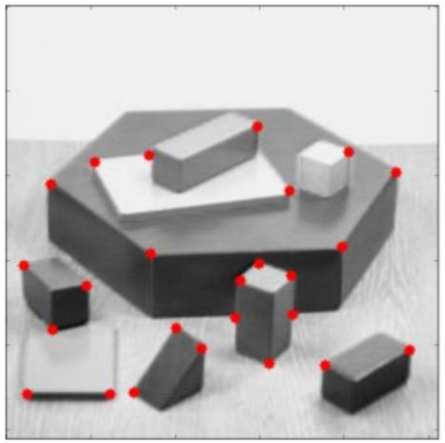
for i in corners:
    x,y = i.ravel()
    cv.circle(img,(x,y),3,255,-1)

plt.imshow(img),plt.show()
```

cv2.goodFeaturesToTrack() parameters:

- **img** - image that should be grayscale.
- **corners** - number of corners you want to find.
- **qualityLevel** - the quality level, which is a value between 0-1 that denotes the threshold for quality of corner
- **minDistance** - the minimum Euclidean distance between corners detected.

The function sorts the computed corners based on quality in the descending order, and takes N strongest corners to return



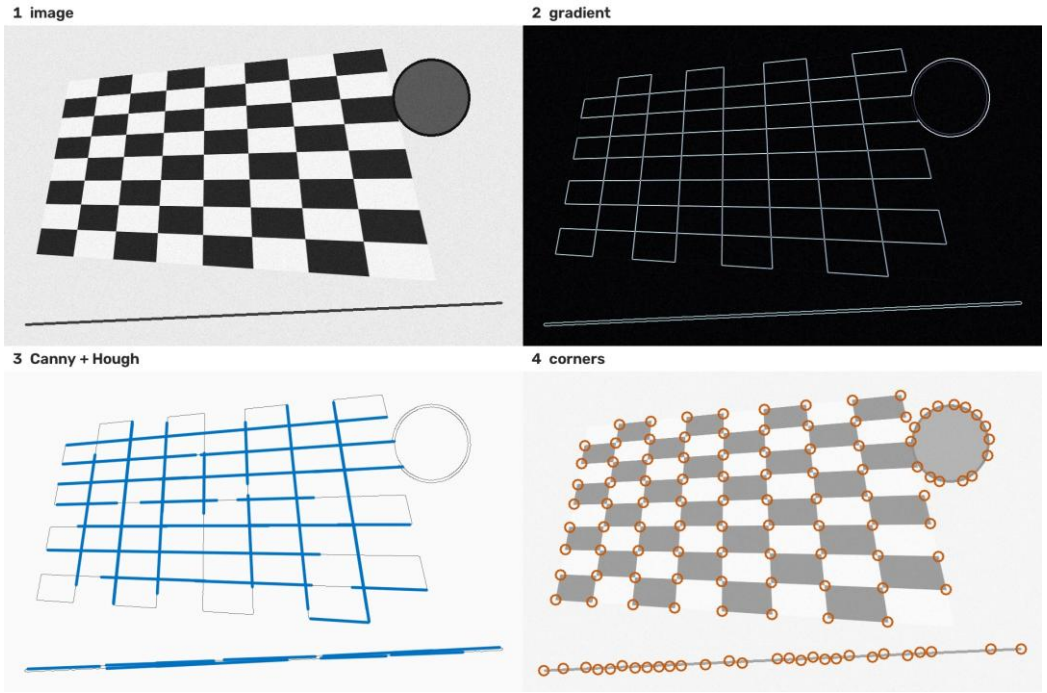
© Shi-Tomasi Corner Detector & Good Features to Track, OpenCV docs, 2022, https://docs.opencv.org/3.4/d4/d8c/tutorial_py_shi_tomasi.html

References

REFERENCES

- Szeliski R. Computer Vision: Algorithms and Applications, 2nd ed., 2022 — §7.2 (края), §7.4 (прямые и робастное оценивание); к L8 — §7.1 (точки интереса)
- Canny J. A Computational Approach to Edge Detection, IEEE TPAMI, 1986 · Duda R., Hart P. Use of the Hough Transform to Detect Lines and Patterns, CACM, 1972
- Fischler M., Bolles R. Random Sample Consensus, CACM, 1981 · Raguram R. et al. USAC: A Universal Framework for Random Sample Consensus, IEEE TPAMI, 2013
- Гонсалес Р., Вудс Р. Цифровая обработка изображений, 3-е изд., 2012 — гл. 9 (морфология), гл. 10 (сегментация по краям)
- OpenCV docs 4.x: `Canny`, `Sobel`/`Scharr`, `findContours`/`approxPolyDP`, `HoughLines`/`HoughLinesP`, `fitLine` (DIST_*), USAC-флаги RANSAC

Вопросы?



Dr. Ilya Afanasyev · i.afanasyev@innopolis.ru